Do Powerful CPUs Reduce the Need for Efficient Code? An Empirical Study of Algorithmic and Memory Performance

Jakob Balkovec

February 26, 2026

Abstract

Modern processors provide dramatically higher computational power than earlier generations of hardware. This raises an important question in software engineering: if hardware continues to become faster, does writing efficient code remain necessary? This short paper investigates that question through empirical experiments evaluating both algorithmic efficiency and memory access patterns. Using repeated benchmark trials, I compared the performance of two sorting algorithms with different computational complexity and examined the impact of cache locality in matrix traversal. The results demonstrate that even on modern hardware, inefficient algorithms and poor memory access patterns produce significant performance penalties. These findings suggest that efficient code remains a fundamental requirement for scalable software systems.

Note

This paper was prepared as part of my graduate application portfolio for the Master's program in Data Engineering and Analytics at the Technical University of Munich.

1 Introduction

Modern CPUs provide enormous computational capability compared to earlier generations of hardware, largely due to advances in semiconductor scaling and processor architecture [11, 4]. Multicore processors, deep cache hierarchies, and aggressive compiler optimizations allow programs to execute billions of instructions per second. Because of these improvements, it is sometimes argued that software developers no longer need to focus heavily on efficiency, since hardware performance can compensate for inefficient implementations.

However, this perspective overlooks important aspects of computational performance. Algorithmic complexity and memory hierarchy continue to impose practical limits on program execution [14]. Even powerful processors cannot overcome fundamental differences in algorithmic growth rates or inefficient memory access patterns.

This paper investigates whether efficient code remains necessary in modern computing environments. Two experiments are conducted. The first compares two sorting algorithms with different computational complexity. The second evaluates how memory access patterns influence runtime through cache locality effects.

2 Background

Performance in modern computing systems depends on several interacting factors, including algorithmic complexity, processor architecture, and memory hierarchy.

Algorithmic complexity describes how the runtime of an algorithm grows as input size increases. Algorithms with quadratic complexity scale dramatically worse than algorithms with near-linear growth rates, particularly for large inputs.

Modern processors rely heavily on hierarchical memory systems consisting of multiple levels of cache and main memory [9]. Accessing data stored in cache is significantly faster than accessing data from main memory. As a result, programs that access memory sequentially often perform substantially better than programs that access memory in irregular patterns [10].

Additionally, performance scaling in modern computing is constrained by fundamental architectural limits such as Amdahl's law and the so-called memory wall [7, 14]. These constraints suggest that efficient algorithms and memory-aware implementations remain critical even as raw computational power increases.

3 Hypotheses

This study evaluates two hypotheses:

- **H1:** Algorithms with better asymptotic complexity maintain significantly lower runtime growth as input size increases.
- **H2:** Memory access patterns that exhibit better cache locality produce significantly lower runtimes than cache-unfriendly patterns, even when algorithmic complexity remains constant.

4 System Configuration

All experiments were conducted on a single Apple M2-based machine in order to maintain consistent hardware conditions across trials. The system uses Apple's M2 system-on-chip architecture with an 8-core CPU composed of four performance cores and four efficiency cores, together with unified memory and 100 GB/s memory bandwidth [1]. These architectural characteristics make the platform appropriate for studying the interaction between algorithmic behavior and memory access patterns.

Modern processors depend heavily on hierarchical memory systems in which fast cache levels serve as intermediaries between the processor cores and main memory. Accessing data already present in cache is substantially faster than retrieving it from main memory, so the order in which programs traverse data can have a measurable effect on runtime [9, 10]. For this reason, the cache-locality experiment in this paper is designed to isolate how memory access order influences performance even when the total amount of work remains fixed.

5 Experimental Setup

All benchmark programs were implemented in C++ and compiled using `g++` with optimization level `-O3`. Runtime measurements were obtained using the C++ `std::chrono` high-resolution clock. Each benchmark was executed repeatedly in order to reduce noise caused by background system activity and operating system scheduling [5].

Input datasets for the sorting experiments were generated using a Python script and written to disk as plain text files. Three dataset sizes were created to evaluate algorithmic scaling, along with three ordering configurations: random order, pre-sorted order, and reverse-sorted order.

The benchmark program loaded each dataset into memory and measured the execution time of the sorting algorithm. Each dataset was tested multiple times, and the resulting runtime measurements were recorded in CSV files for later analysis.

A second benchmark was implemented to evaluate memory locality. This experiment performed traversal of large matrices using two access patterns: row-major traversal and column-major traversal. Both approaches perform the same number of arithmetic operations but differ in how memory is accessed. Because modern CPUs rely heavily on cache hierarchies, sequential memory access patterns are expected to exhibit better performance due to improved spatial locality [9, 10].

6 Experiment 1: Algorithmic Efficiency

The first experiment evaluates the effect of algorithmic complexity on runtime performance by comparing two sorting algorithms with fundamentally different growth rates.

The first algorithm is Bubble Sort, a simple comparison-based method that repeatedly scans the array and swaps adjacent elements that are out of order [8]. In the worst case, the algorithm performs a sequence of passes over the dataset, where each pass compares one fewer pair of elements than the previous one. The total number of comparisons can therefore be written as

$$\sum_{i=1}^{n-1} i,$$

which is the sum of the first $n-1$ positive integers. Using the closed-form expression for an arithmetic series,

$$\sum_{i=1}^{k} i = \frac{k(k+1)}{2},$$

and substituting $k = n-1$, we obtain

$$\frac{n(n-1)}{2}.$$

This leads directly to a time complexity of $O(n^2)$, meaning that the number of required operations

grows proportionally to the square of the input size [8]. As n increases, this growth rate causes runtime to rise rapidly, making Bubble Sort impractical for large datasets.

The second algorithm tested is the C++ standard library implementation `std::sort`. Modern implementations of `std::sort` use a hybrid algorithm known as *introsort*, originally proposed by Musser [12]. Introsort combines multiple sorting strategies in order to achieve both strong theoretical guarantees and high practical performance [12, 3].

Introsort begins with quicksort, a divide-and-conquer algorithm that partitions the input around a pivot and recursively sorts the resulting subarrays. When these partitions remain approximately balanced, the recurrence relation can be written as

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

where the linear term represents the cost of partitioning at each level of recursion. This recurrence fits the standard form of the Master Theorem,

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

with $a = 2$, $b = 2$, and $f(n) = O(n)$. Evaluating the comparison term gives

$$n^{\log_b a} = n^{\log_2 2} = n.$$

Because $f(n) = \Theta(n)$, the recurrence falls into Case 2 of the Master Theorem, yielding

$$T(n) = \Theta(n \log n)$$

for the average-case runtime of quicksort [2, 13]. This asymptotic difference is substantial:

$$n \log n \ll n^2 \quad \text{as } n \to \infty,$$

so algorithms with $O(n \log n)$ complexity scale far more effectively than quadratic algorithms as input size increases.

Quicksort alone, however, does not guarantee this behavior in the worst case. If pivot selection repeatedly produces highly unbalanced partitions, the recurrence becomes

$$T(n) = T(n-1) + O(n),$$

which yields worst-case complexity $O(n^2)$ [8]. To avoid this degeneration, introsort monitors recursion depth and switches to heapsort once the depth exceeds a threshold proportional to

$$2 \log_2 n.$$

Since heapsort guarantees $O(n \log n)$ worst-case complexity, this transition ensures that the overall algorithm never degrades to quadratic behavior [12]. In addition, very small subarrays are typically handled using insertion sort, whose low constant overhead makes it efficient for small partitions [2, 13].

Because of this hybrid design, `std::sort` combines strong asymptotic guarantees with excellent practical performance. The purpose of this experiment is to evaluate how these theoretical differences appear in practice. Both algorithms were executed on identical datasets while varying the input size. By comparing the observed runtime growth of Bubble Sort and `std::sort`, the experiment tests whether improvements in modern processor performance reduce the practical importance of algorithmic efficiency.

6.1 Results

Table 1: Average runtime of Bubble Sort and `std::sort` for randomly ordered datasets.

Input size n	Bubble Sort (s)	`std::sort` (s)	Slowdown
1,000	0.0288	0.000163	176×
10,000	1.9004	0.000855	2,223×
100,000	8.2739	0.001727	4,792×

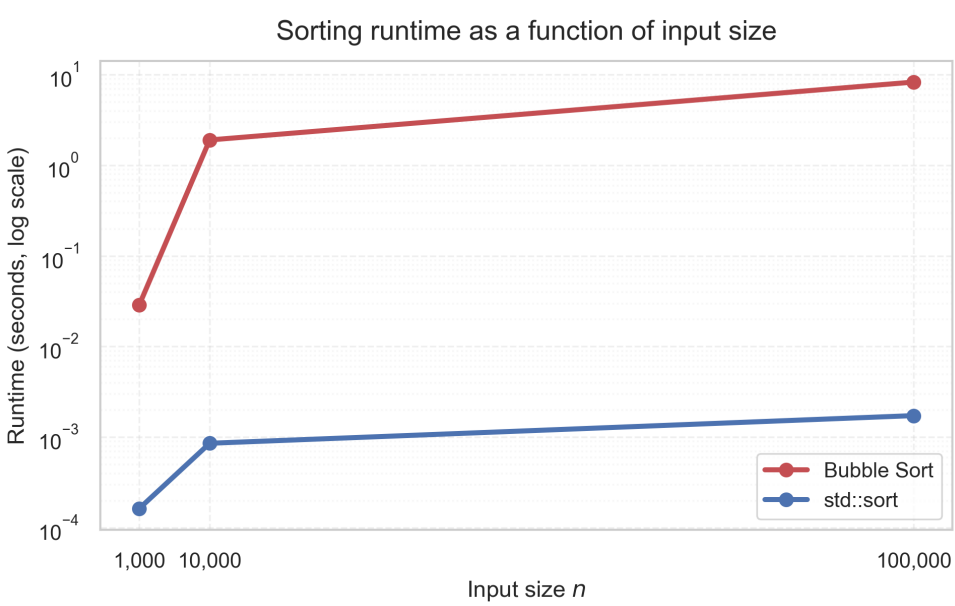

Figure 1: Sorting runtime as a function of input size n.

Figure 1 shows a clear divergence in runtime growth between the two algorithms as input size increases. For random datasets of size $n = 1{,}000$, Bubble Sort required an average runtime of 2.88×10^{-2} seconds, compared to 1.63×10^{-4} seconds for `std::sort`. When the dataset size

increased to $n = 10{,}000$, the runtime of Bubble Sort rose to approximately 1.90 seconds, while `std::sort` required only 8.55×10^{-4} seconds. For the largest dataset tested ($n = 100{,}000$), Bubble Sort required 8.27 seconds on average, whereas `std::sort` completed in approximately 1.73×10^{-3} seconds.

These results show that runtime grows dramatically faster for Bubble Sort as input size increases, whereas the growth of `std::sort` remains comparatively modest. This behavior is consistent with the theoretical difference between quadratic $O(n^2)$ and $O(n \log n)$ complexity.

7 Experiment 2: Cache Locality

The second experiment evaluates the impact of memory access patterns on runtime performance. While the previous experiment examined differences in algorithmic complexity, this experiment isolates a lower-level implementation factor: how programs interact with the processor's memory hierarchy.

Modern processors rely heavily on hierarchical memory systems consisting of multiple cache levels positioned between the CPU and main memory. Accessing data from cache is significantly faster than retrieving it from main memory, so programs that reuse nearby memory locations tend to perform better due to improved spatial locality [9, 10, 6]. As a result, the order in which data is traversed can have a measurable impact on runtime even when the number of arithmetic operations remains identical.

To examine this effect, two traversal strategies were compared:

- Row-major traversal
- Column-major traversal

Both implementations perform the same total number of operations but differ in the order in which elements are accessed.

In most C and C++ implementations, two-dimensional arrays are stored in *row-major order*. For a matrix A with n rows and m columns, the memory address of element $A[i, j]$ can be expressed as

$$\mathrm{addr}(A[i, j]) = B + (i \cdot m + j)\, s,$$

where B is the base address of the array and s is the size of each element in bytes [6]. In this representation, consecutive elements of the same row occupy adjacent memory locations.

Row-major traversal accesses the matrix in the order

$$A[0, 0], A[0, 1], A[0, 2], \ldots$$

which corresponds to sequential memory accesses. Because modern CPUs fetch memory in fixed-size blocks known as cache lines [6], typically containing multiple adjacent elements, sequential access allows many operations to reuse data already present in cache.

Column-major traversal, in contrast, accesses elements in the order

$$A[0,0], A[1,0], A[2,0], \ldots$$

which produces a memory stride proportional to the row length. If each row contains m elements, the effective stride between consecutive accesses becomes

$$\text{stride} = m \cdot s.$$

When this stride exceeds the size of a cache line, each access is more likely to require fetching a new memory block from lower levels of the memory hierarchy. This behavior increases cache misses and significantly increases effective memory latency [9, 10].

Importantly, both traversal strategies perform the same number of arithmetic operations and therefore have identical asymptotic complexity. The only difference lies in how the program interacts with the memory hierarchy. This makes the experiment a useful demonstration that implementation details (particularly memory locality) can strongly influence runtime performance even when algorithmic complexity remains unchanged.

7.1 Results

Table 2: Average runtime for row-major and column-major matrix traversal.

Matrix size N	Row-major (s)	Column-major (s)	Slowdown
1024	0.001017	0.002527	2.49×
2048	0.004204	0.017667	4.20×
4096	0.016051	0.092735	5.78×

Figure 2 shows that row-major traversal consistently outperformed column-major traversal across all tested matrix sizes. For a matrix of dimension $N = 1024$, row-major traversal required approximately 1.02×10^{-3} seconds on average, whereas column-major traversal required 2.53×10^{-3} seconds. When the matrix dimension increased to $N = 2048$, runtimes increased to 4.20×10^{-3} seconds for row-major traversal and 1.77×10^{-2} seconds for column-major traversal. For the largest matrix tested ($N = 4096$), row-major traversal required 1.61×10^{-2} seconds, while column-major traversal required 9.27×10^{-2} seconds.

Across all tested sizes, column-major traversal was consistently slower, reflecting the reduced cache efficiency associated with strided memory accesses.

These results align with prior studies of memory hierarchy behavior, which demonstrate that spatial locality and cache-aware access patterns are critical factors in achieving high performance on modern architectures [9, 10, 6].

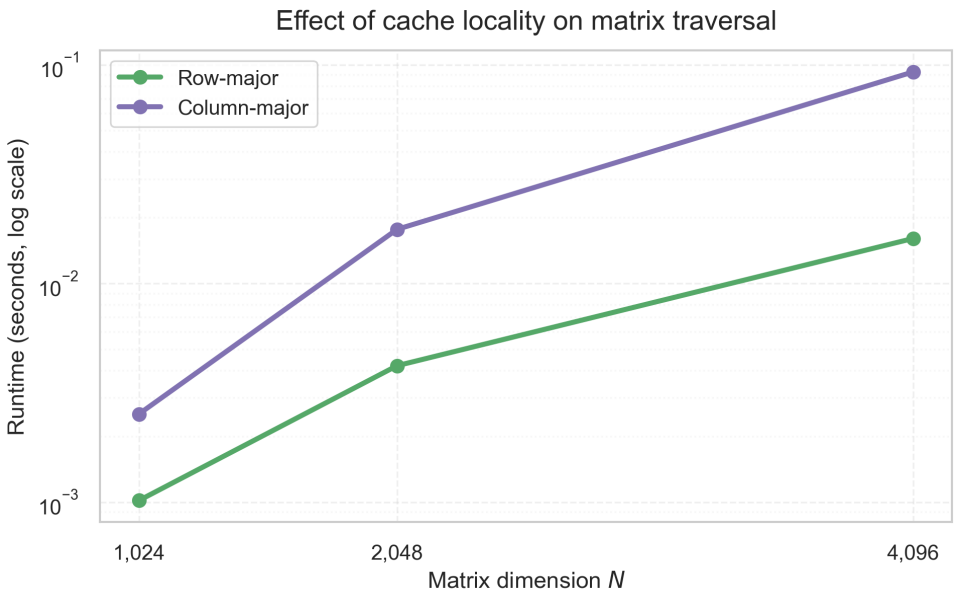

Figure 2: Runtime comparison between row-major and column-major traversal of an $N \times N$ matrix.

8 Discussion

The experimental results demonstrate that improvements in modern processor performance do not remove the need for efficient algorithms or memory-aware implementations. Instead, hardware improvements primarily change constant factors in execution time while leaving the underlying growth behavior determined by algorithmic complexity and memory access patterns.

The sorting experiment illustrates the continued importance of algorithmic complexity. For randomly ordered datasets of size $n = 1{,}000$, Bubble Sort required approximately 2.88×10^{-2} seconds on average, compared to 1.63×10^{-4} seconds for `std::sort`. As the dataset size increased to $n = 10{,}000$, Bubble Sort required approximately 1.90 seconds while `std::sort` required 8.55×10^{-4} seconds. For the largest dataset tested ($n = 100{,}000$), Bubble Sort required 8.27 seconds compared to 1.73×10^{-3} seconds for `std::sort`.

These measurements correspond to performance differences of approximately 176×, 2,223×, and 4,792×, respectively. The widening gap between the algorithms closely reflects the theoretical difference between quadratic $O(n^2)$ and $O(n \log n)$ growth. Even on modern processors capable of executing billions of instructions per second, inefficient algorithms rapidly become impractical as input sizes increase.

The cache locality experiment highlights a second dimension of performance that is not captured by asymptotic complexity alone. Both traversal strategies perform the same number of arithmetic operations and therefore share identical theoretical complexity $O(n^2)$ for traversing an $N \times N$ matrix. However, the experiments reveal substantial runtime differences caused solely by memory access patterns.

For $N = 1024$, column-major traversal was approximately 2.49× slower than row-major traversal. This slowdown increased to roughly 4.20× for $N = 2048$ and 5.78× for $N = 4096$. The increasing

performance gap reflects the growing cost of cache misses as the working set size exceeds the capacity of faster cache levels. Because column-major traversal accesses memory with a stride proportional to the row length, consecutive accesses often fall into different cache lines, reducing spatial locality and increasing memory latency.

Taken together, these experiments illustrate two complementary principles of performance engineering. First, algorithmic complexity determines how runtime scales with input size. Second, the interaction between software and hardware architecture (particularly memory hierarchy behavior) can significantly affect performance even when algorithmic complexity remains unchanged. Modern hardware therefore does not eliminate the importance of efficient code; rather, it makes understanding both algorithm design and architectural behavior increasingly important.

9 Threats to Validity

Several limitations should be considered when interpreting these results. Benchmark timings may be influenced by background system activity, operating system scheduling, and other sources of runtime noise that are difficult to eliminate entirely. Although multiple trials were conducted in order to reduce this variability, small fluctuations in measured runtime may still occur.

Additionally, the experiments were conducted on a single hardware platform. Modern processors vary widely in their cache sizes, memory bandwidth, and microarchitectural design. As a result, the absolute runtime values observed in this study may differ on other systems with different cache hierarchies or memory subsystems.

The sorting experiments also used a limited set of dataset configurations (random, sorted, and reverse-sorted inputs). While these cases capture common input patterns, other distributions could influence the behavior of specific sorting algorithms.

Finally, all programs were compiled using a single compiler configuration. Different compilers, optimization levels, or standard library implementations may produce slightly different performance characteristics.

Despite these limitations, the observed trends align closely with well-established principles in computer architecture and algorithmic analysis. The results therefore provide a useful empirical illustration of how algorithmic complexity and memory locality affect performance on modern hardware.

10 Conclusion

This study investigated whether improvements in modern CPU performance reduce the need for efficient code. Through empirical experiments examining both algorithmic complexity and memory access patterns, the results demonstrate that efficient implementations remain essential for scalable software systems.

The sorting experiment showed that algorithms with different asymptotic complexity diverge dramatically in practice as input sizes grow. Across the tested datasets, the performance gap between Bubble Sort and `std::sort` increased from hundreds to several thousand times as the

input size increased. This behavior closely reflects the theoretical difference between quadratic $O(n^2)$ and near-linear $O(n \log n)$ growth.

The cache locality experiment further demonstrated that implementation details can significantly affect performance even when algorithmic complexity remains identical. Traversing matrices in a memory-friendly row-major order consistently outperformed column-major traversal, with slowdowns exceeding fivefold for larger matrices due to reduced cache efficiency.

Taken together, these results illustrate that improvements in processor speed do not eliminate the importance of efficient algorithms and memory-aware programming. Instead, they reinforce the need for software that is designed with both algorithmic complexity and hardware architecture in mind. As modern systems continue to scale in both data size and computational demand, understanding these principles remains fundamental to achieving high performance.

References

[1] Apple Inc. Macbook pro (13-inch, m2, 2022) – technical specifications. `https://support.apple.com/en-us/111869`, 2022. Accessed March 14, 2026.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.

[3] cppreference.com. std::sort. `https://en.cppreference.com/w/cpp/algorithm/sort.html`, 2025. Accessed March 14, 2026.

[4] Robert Dennard. Design of ion-implanted mosfets with very small physical dimensions. *IEEE Journal of Solid-State Circuits*, 1974.

[5] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *OOPSLA*, 2007.

[6] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

[7] Mark D Hill and Michael R Marty. Amdahl's law in the multicore era. *IEEE Computer*, 2008.

[8] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 1998.

[9] Monica S Lam, Edward E Rothberg, and Michael E Wolf. The cache performance and optimizations of blocked algorithms. *ACM SIGPLAN Notices*, 1991.

[10] Sally A McKee. Reflections on the memory wall. *Proceedings of the conference on computing frontiers*, 2004.

[11] Gordon Moore. Cramming more components onto integrated circuits. *Electronics*, 1965.

[12] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.

[13] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, Boston, 4 edition, 2011.

[14] William A Wulf and Sally A McKee. Hitting the memory wall. *ACM SIGARCH Computer Architecture News*, 1995.